



INDIVIDUAL CAPSTONE

REPORT

Title: Development of a Mobile-based Adaptive Academic Learning System for Personalized study and Revision using Learning Analytics and Gamification

Date:

April 3rd, 2026



Table of Contents

A. General Information	6
CHAPTER 1: Project Introduction	1
1.1 Executive Summary	1
1.2 Project Objectives	1
1.3 Project Description.....	1
1.4 Project Scope	2
1.4.1 Inclusions	2
1.4.2 Exclusions	2
1.5 Project Organisation.....	2
1.6 Summary of Milestones and Deliverables	2
CHAPTER 2: Related Works	4
2.1 Overview of Similar Systems and Prior Research	4
2.2 Methodology of the Literature Review	4
2.3 ETL Pipeline Design and Data Engineering for Price Information Systems	5
2.4 Data Quality, Missing Value Imputation, and Preprocessing	6
2.5 Relational Database Design for Structured Price Data Storage	6
2.6 Agricultural Price Forecasting: Model Selection and Comparative Analysis.....	7
2.7 Summary Literature Review	8
CHAPTER 3: System Analysis	10
3.1 Theoretical Analysis: SWOT	10
3.1.1 Python	10
3.1.2 Pandas	11
3.1.3 NumPy	13
3.1.4 Statsmodels	14
3.1.5 Flask.....	15
3.1.6 SQLAlchemy	16
3.1.7 PostgreSQL	17
3.2 Technical Analysis.....	19

3.2.1 Data Pipeline Architecture	19
3.2.2 Analytical Methods	19
3.2.3 Mock Data Generation Strategy (Capstone 1 Prototype).....	20
3.3 Requirement Specifications	21
3.3.1 Functional Requirements	21
3.3.2 Non-Functional Requirements	22
3.4 Summary of System Analysis	23
CHAPTER 4: System Design	25
4.1 System Architecture Diagram	25
4.2 Data Flow Diagram.....	26
4.3 Database Schema Diagram	26
4.4 Use Case Diagram.....	27
4.5 Class Diagram.....	28
4.6 Activity Diagram	29
4.7 Sequence Diagram	30
4.8 Pseudocode	31
4.8.1 ETL Pipeline – DataPipeline.run().....	31
4.8.2 ARIMA Forecasting – AnalyticsEngine.forecast().....	32
4.8.3 Mock Data Generator – Capstone 1 Prototype	33
4.9 Summary of System Design.....	34
References.....	36
Appendix.....	38
Appendix A: Project Gantt Chart.....	38
Appendix B: Excalidraw Link:	38
Appendix C: Meeting Minutes.....	39

Table of Figures

Figure 1: System Architecture Diagram	26
Figure 2: Data Flow Diagram	26
Figure 3: Database Schema Diagram.....	27
Figure 4: Use Case Diagram	28
Figure 5: Class Diagram	29
Figure 6: Activity Diagram.....	30
Figure 7: Sequence Diagram.....	31
Figure 8: Gantt Chart	38

Table of Tables

Table 1: Project Organisation	2
Table 2: Summary of Milestones and Deliverables	2
Table 3: Python SWOT Analysis.....	10
Table 4: Pandas SWOT Analysis.....	12
Table 5: Numpy SWOT Analysis	13
Table 6: Statsmodels SWOT Analysis.....	14
Table 7: Flask SWOT Analysis	15
Table 8: SQLAlchemy SWOT Analysis.....	16
Table 9: PostgreSQL SWOT Analysis.....	18
Table 10: Functional Requirements	21
Table 11: Non-functional Requirements.....	22

A. General Information

Position	Full Name	Phone	Email
Supervisor	Rabin Thapa		Rabin@iimscollege.edu.np
Project Lead / Data Engineer	Subrna Thakurathi	9749310885	Subrna2080-0236@iimscollege.edu.np
Backend Engineer	Robinson Ghimire	9811094407	Robinson2080-0278@iimscollege.edu.np
Frontend Developer	Ayush Bist	9749752434	Ayush2080-0275@iimscollege.edu.np
Analytics Lead	Nikit Shrestha	9803004211	Nikit2080-0276@iimscollege.edu.np
Database Developer	Shashi Thapa Magar	9860326137	Shashi2080-0203@iimscollege.edu.np

CHAPTER 1: Project Introduction

Note: This chapter was submitted in full as part of the Group Capstone Proposal (Group 24, February 6, 2026). As per individual report guidelines, a summarised version relevant to the role of Data Engineer and System Architect is presented here.

1.1 Executive Summary

The present capstone project suggests a web-based price monitoring, analyzing, and predicting system at the Kalimati Fruits and Vegetable Wholesale Market, Kathmandu. The issue is simple: the daily price information is publicly accessible but in raw and unstructured format and without the history analysis, trend visibility, or ability to predict. The project attempts to solve this by creating an entire data pipeline and analytics infrastructure on the Kalimati open-source GitHub data. The project implementation is in two stages. Capstone 1 is dedicated to system design, architecture, and a working React-based frontend prototype backed by fake data - illustrating all three fundamental views: price trend analysis with 7-day moving average, cross commodity volatility analysis and 30-day forecasting visualization with MAE and RMSE measures. Capstone 2 will be used to finalize the backend Python data pipeline, SQL database integration and ARIMA based forecasting on actual Kalimati data. My role as Project Lead and Data Engineer will be to ensure general coordination, system architecture, and information pipeline which will support the entire system in Capstone 2.

1.2 Project Objectives

Relevant to my role:

- Gather and combine historical prices data of the Kalimati open-source GitHub repository. (ErKiran/kalimati).
- Normalize, clean and preprocess vegetable prices in order to make them analytically accurate.
- Create a time-series optimized relational SQL database schema.
- Construct a modular data pipeline which feeds in processed data into the analytics and the visualization layers.
- Integration of coordinate system of all team components, including data layer to frontend.

1.3 Project Description

The system is a full stack web app with layered modular architecture. The Capstone 1 deliverable is a prototype React (Vite) frontend using programmatically generated mock data to simulate realistic Kalimati price behaviour, through a random-walk with mean reversion of 8 commodity types over 180

days. The prototype covers commodity filtering, date-range selection, moving averaging, volatility ranking and displaying forecasts - illustrating the full desired user experience, before connecting the prototype with the backend, in Capstone 2.

1.4 Project Scope

1.4.1 Inclusions

Designing system architecture, React frontend prototype based on mock data, SQL database schema design (Capstone 2), Python ETL data pipeline (Capstone 2), ARIMA forecasting module (Capstone 2), and project coordination.

1.4.2 Exclusions

Capstone 1 does not include live backend and real Kalimati data integration, ARIMA model deployment, or mobile application development and transfers to Capstone 2. The complete project does not include any external sources of data other than the Kalimati GitHub repository, the real time streaming, and user authentication.

1.5 Project Organisation

Table 1: Project Organisation

Full Name	Role	Key Responsibilities
Subrna Thakurathi	Data Engineer & System Architect (Lead)	Architecture, ETL pipeline, data cleaning, coordination
Robinson Ghimire	Backend & Optimisation Engineer	Flask APIs, database integration, performance optimisation
Ayush Bist	Frontend & Visualisation Developer	React UI, interactive dashboards, chart implementation
Nikit Shrestha	Analytics & Algorithm Lead	Trend analysis, volatility, ARIMA forecasting
Shashi Thapa Magar	Database Developer	Schema design, indexing, query optimisation

1.6 Summary of Milestones and Deliverables

Table 2: Summary of Milestones and Deliverables

No.	Milestone	Responsible	Duration
1	Dataset collection, cleaning & preprocessing	Subrna Thakurathi	10 days
2	Database design & data storage	Shashi Thapa Magar	7 days
3	Trend analysis & volatility detection module	Robinson Ghimire	10 days
4	Price forecasting & prediction module	Nikit Shrestha	8 days
5	Frontend prototype & UI development	Ayush Bist	10 days
6	System testing & documentation	Full Team	7 days

The 50-day cycle is divided into Capstone 1 (Milestones 1- 2 design and Milestone 5 prototype) and Capstone 2 (Milestones 3-4 and full system integration). Milestone 5 (Capstone 1 frontend prototype) is the main submission artifact in this semester to illustrate the proposed UI of the system, visualization design, and interaction flow before this system is fully developed in the backend.

CHAPTER 2: Related Works

2.1 Overview of Similar Systems and Prior Research

The Kalimati Fruits and Vegetable Wholesale Market is one of the richest, yet under-analysed, market contexts within Nepal. The price data are compiled, recorded, and published daily, have been available for many years, and have been made accessible to all as a publicly available open-source CSV file in a GitHub repository (ErKiran/kalimati). However, an integrated system for structured data ingest, long-term storage, multidimensional analysis, and interactive visualization does not yet exist within a single, deployable unit. This chapter reviews the twelve primary and two supplementary (foundation) sources-categorized into four thematic clusters-upon which this project's technical structure, data engineering choices, analytical methods and overall system design is predicated.

The Capstone 1 submission is a functional frontend prototype based on React and powered by programmatically generated mock data. The prototype demonstrates the three system views described above: the historical price trend chart with a moving average overlay, the ranked cross-commodity volatility comparison, and the 30-day forward-looking forecasts with error measure indications. Thus, this is a more concrete deliverable than a static wireframe or Figma mock-up. This methodology for frontend testing against simulated data prior to the integration of the backend data infrastructure is supported by best practice throughout the literature base consulted for this project. It is important to distinguish between the simulation offered by Capstone 1 and the actual forecast produced by Capstone 2, as explained below.

2.2 Methodology of the Literature Review

The literature review for this study employed a purposive, thematic selection technique grouped under 4 areas directly linked to the project architecture and technology: (1) Market information systems architecture - ETL pipeline design & data engineering; (2) Time-series data imputation and pre-processing and data quality; (3) Structured price data storage- relational database design and normalisation; and (4) Market information systems and agricultural price forecasting- developing countries context (Nepal).

The sources were collected from Google Scholar, IEEE Xplore, SpringerLink, ScienceDirect, MDPI Open Access, ResearchGate and the ACM Digital Library. Search strings included; "ETL pipeline agricultural data", "data cleaning time series missing values", "relational database normalisation", "ARIMA vegetable price forecasting Nepal", "SARIMA LSTM agricultural price prediction", "Kalimati market price analysis", "digital agricultural information systems developing countries", "data engineering pipeline design", "time series imputation benchmarking". Literature was searched between January 2026 and March 2026.

All literature had to meet criteria (a), (b) and (c): (a) the sources needed to be either peer-reviewed journal articles, conference proceedings or highly cited technical reference text; (b) the source needed to have been published in reputable publications such as IEEE, Springer, MDPI, Elsevier, Cogent, ACM or SciPy Proceedings; and (c) the source needed to be directly related to at least one of the four themes stated above. Sources older than 2016 were included as early influential references where relevant; Rahm & Do (2000) for data cleaning, and Vassiliadis (2009) for ETL technology, both remain highly cited within data engineering literature. Source excluded were only front-end focused development (i.e. Mobile applications) or pure market economics, without a focus on infrastructure and data. There are fourteen sources: twelve that are project related, and two sources which were added in this chapter to help develop the data engineering/ETL foundation.

2.3 ETL Pipeline Design and Data Engineering for Price Information Systems

The core difficulty is not with forecasting models per se, but rather with the design and robustness of a data engineering infrastructure capable of taking raw market data and translating it into analytically consumable outputs. Vassiliadis (2009) offers a complete survey of Extract-Transform-Load (ETL) technology published in the International Journal of Data Warehousing and Mining, defining it as a process where "data is extracted from the data sources, transformed using cleansing and integration processes and finally loaded into a target data structure for analysis and reporting." He describes three inherent failure modes within any ETL process, each of which directly translates to the challenges encountered within the Kalimati pipeline: Schema heterogeneity between sources and target. Data quality violations occurring during the extraction phase. Transformation performance issues due to complex or poorly-written business logic.

The Kalimati CSV dataset itself exhibits all three of these issues: variation in commodity name spelling across multiple yearly files (schema heterogeneity), absence of particular data points and formatting problems within files (data quality violations), and a need to reconstruct multi-year time series for individual commodities (transformation complexity). The design for this ETL architecture-which consists of modular ingestion, name normalization, gap detection, imputation, validation, and batched loading-mirrors the structured approach of the ETL process as laid out by Vassiliadis (2009). Rana et al. (2024) constructed a similar big-data ETL pipeline over a 2007–2022 time series of Pakistani commodity prices in an effort to build predictive models (ARIMA, random forest, and LSTM). Their chief finding was that the pre-processing phase-in particular, managing missing records, standardizing price scaling, and consolidating time series into consistent temporal formats-was the most crucial determinant of model performance comparison. Even top forecasting algorithms could yield unusable results when applied to unstructured, badly pre-processed data.

This supports the paramount importance given to the ETL pipeline in the current project, and explains why the DataPipeline class is treated not just as a data ingestion step but as a quality assurance check.

Mouratiadou et al. (2023) outline their Digital Agricultural Knowledge and Information System (DAKIS), a multi-component system linking sensor networks, AI algorithms, public databases, and user interfaces within an integrated decision support platform. Their overall system architecture (structured backend data store read by an analytics module, which in turn feeds a front-end user interface) closely maps onto the 5-layered architecture designed for this capstone, and their discussion regarding difficulties in implementing even a basic digitally integrated system reinforces the engineering challenge presented by this capstone. Their discussion of modularity and stakeholder-driven design also reflects a similar architectural decision-making philosophy to that of the current project. Choong et al. (2021) also conducted a review of SARIMA and GANN models for forecasting vegetable prices on e-commerce platforms and came to the same conclusion: that complex forecasting methods do not perform as well when applied to poorly organized historical data, emphasizing the importance of good data management infrastructure.

2.4 Data Quality, Missing Value Imputation, and Preprocessing

Rahm and Do (2000), in a widely-cited IEEE Data Engineering Bulletin article, provide a systematic taxonomy of data quality problems applicable to structured datasets, categorising them as single-source problems (missing values, outliers, inconsistent formats) and multi-source problems (naming conflicts, schema heterogeneity). Their analysis of imputation strategies — including ignore, manual correction, global constant imputation, and advanced statistical methods — establishes the conceptual framework within which the Kalimati preprocessing pipeline was designed. Critically, Rahm and Do (2000) demonstrate that the choice of imputation method is not arbitrary: applying a statistically inappropriate method can introduce systematic bias that distorts downstream analytical results more severely than the original missing values. This finding directly informs the tiered imputation strategy in this project: forward-fill for gaps of three or fewer days, linear interpolation for gaps up to fourteen days, and flagging with pipeline logging for longer gaps — each tier chosen to preserve the temporal integrity of the price series while avoiding the introduction of artificial trends.

Kazijevs and Samad (2023), in the Journal of Biomedical Informatics, provide a comprehensive benchmarking study of deep imputation methods for multivariate time-series data. While their context is health informatics, their core methodological finding generalises directly to commodity price series: cross-sectional imputation methods that ignore the temporal ordering of observations consistently underperform longitudinal methods that respect time dependencies. For the Kalimati dataset — where price records for a given commodity form a continuous daily time-series with clear temporal autocorrelation — this result confirms that forward-fill and linear interpolation, which both preserve temporal order and smooth over short gaps without introducing non-existent trend changes, are the methodologically appropriate choices over mean substitution or model-based cross-sectional imputation.

2.5 Relational Database Design for Structured Price Data Storage

The work of Rahm and Do (2000) is a foundational survey that defines a general classification for data quality problems for structured data-distinguishing between single-source problems such as missing values, outliers, and format inconsistencies and multi-source problems such as name clashes and schema heterogeneity. Their evaluation of various imputation techniques, from those simply ignoring values and manually correcting errors to globally constant, globally averaging and more complex statistical methods, provides the basis for the imputation strategies chosen for the Kalimati pre-processing pipeline. It is of particular note that Rahm and Do (2000) argue that an improperly chosen statistical method for imputation is capable of introducing a systemic bias into the data that corrupts the results of the subsequent analysis much more severely than do original missing values. This finding directly motivates the use of multi-tiered imputation in this project where gap length determines whether forward-filling up to 3 days, linear interpolation up to 14 days or merely tagging and logging with the pipeline occurs, each approach preserving the temporal nature of the time-series data while avoiding the addition of artificial trends.

The paper by Kazijevs and Samad (2023), published in the Journal of Biomedical Informatics, performs a benchmark study of deep imputation methods for multivariate time-series data. Their research on health informatics time-series yields findings that translate directly to this work: imputation methods based on cross-sectional imputation will consistently perform worse than methods that treat the data longitudinally. The commodity prices themselves, for any given commodity, are ordered by time, creating a daily time-series that exhibits temporal autocorrelation. In such data, methods such as forward-fill and linear interpolation are appropriate in comparison to simple mean substitution or cross-sectional imputation by statistical model because they take the temporal sequence of data into account.

2.6 Agricultural Price Forecasting: Model Selection and Comparative Analysis

The first literature cluster, at the forecasting level, clearly delineates a trade-off between statistical interpretability and predictive complexity. The most geographically-relevant paper, by Giri and Giri (2024) published in Cogent Food and Agriculture, compares SARIMA, LSTM and Facebook Prophet for forecasting cauliflower prices in Nepal and importantly concludes that SARIMA significantly outperformed the more complex AI models on the Nepali cauliflower data. This confirms that model complexity does not necessarily translate to accuracy with a smaller dataset with regular seasonality and no significant non-linear dependencies. This is the fundamental precedent for using ARIMA as the initial model, i.e., working backwards from the simple to the complex.

Working directly with the Kalimati dataset, Xia (2024) compares ARIMA and LSTM to forecast Kalimati market prices and concludes that ARIMA reflects market volatility better for generally stable price trends. This is perhaps the single most important citation in this project's literature review; the choice to use ARIMA instead of LSTM is thus not due to limitations on available resources but methodological accuracy for the given dataset and forecast horizon. This conclusion is confirmed by Udari and

Hemachandra (2024) who model bean and tomato prices in Sri Lanka in the Journal of Technology and Value Addition; they conclude that the ARIMA(2,1,1) is the best short-term model, and also highlight the fact that wholesale prices show greater volatility than retail – the Kalimati data is at the wholesale level. RECPHEC (2016), the foundational Nepali market study, contextualizes the problem on the ground: "The current information system provides daily lists of prices with no historical analysis and no volatility monitoring and the ability to forecast future trends has not been incorporated".

At the cutting edge of the literature, Bao et al. (2025) proposes CNN-LSTM-Attention architectures, and Zhao et al. (2025) uses an LLM-based Mixture of Experts system; these models offer state-of-the-art accuracy but demand significantly greater computational resources and large training datasets that go beyond the scope of this student-led capstone project.

Together, the works of Xia (2024) and Giri and Giri (2024) strongly suggest that these advanced models offer declining returns for a small, localized agricultural dataset, supporting the choice to limit the model scope to ARIMA. Zhang et al. (2024) provides strong results for vegetable prices in Beijing using LSTM but uses a significantly larger dataset than that available in Kalimati. Dinesh Kumar (2025) promotes the practicality of Facebook Prophet due to its speed and ease of dealing with missing values, however the team prefers the statistical interpretability of ARIMA, especially given Xia's (2024) success using ARIMA on the Kalimati data. Subhasree and Arun Priya (2016) proposed early benchmarks for neural networks on vegetable price forecasting, concluding a GANN achieves 89% accuracy, however again the required training dataset size was much greater than the Kalimati data allows for.

The research identified within this thematic cluster strongly support three key design choices for this project: to use ARIMA as the primary model; to employ Augmented Dickey-Fuller test to check for stationarity prior to model fitting; and to extend to SARIMA if it is determined the data is seasonal non-stationary in the Capstone 2 analysis phase. This seasonal forecasting option is further supported by the work of Ramesh and Khosla (2019), which shows that Seasonal ARIMA accurately forecasts fruit and vegetable prices under significant inflation in Indian markets, which bears structural similarities to the Nepalese economy.

2.7 Summary Literature Review

The fourteen sources I reviewed across the four themed clusters show an undeniable, resounding consensus: the performance bottleneck of agricultural price intelligence systems is not in the prediction algorithm but in the data engineering pipeline that processes raw transactions to an analytical format. This point was further corroborated by Vassiliadis (2009) for ETL literature independently. The same was mentioned for agricultural data pipelines by Rana et al. (2024) and in the review for e-commerce price forecasting literature by Choong et al. (2021) and for digital agriculture by Mouratiadou et al. (2023). This is the theoretical justification for focusing on the data pipeline in this project's architecture.

The data quality literature, consisting of Rahm and Do (2000) and Kazijevs and Samad (2023), frames the tiered imputation methodology used in the data pipeline: daily commodity price series are subject to temporal dependency, and only time-aware imputation can be used, and an imputation method is not neutral but causes its own distortion of subsequent analysis. The literature frames the relational schema using normalisation theory (Akadal and Satman, 2022): normalized relation databases are suitable for the normalized structure inherent in the Kalimati price records and are the most efficient option. The forecasting literature shows the evolution from early neural networks (Subhasree and Arun Priya, 2016) through benchmarks with ARIMA (Ramesh and Khosla, 2019; Udari and Hemachandra, 2024) to modern state-of-the-art deep learning and LLM models (Bao et al., 2025; Zhao et al., 2025). The most relevant geographic studies, however, show that in localizing data for the Nepali agricultural market (Giri and Giri, 2024; Xia, 2024), ARIMA is more accurate than complex models. The fusion of geographic relevance with performance supports the use of ARIMA as a benchmark model for this project.

Collectively, the fourteen sources show the absence of one complete system that merges structured data engineering pipeline design, normalized relational databases, multidimensional analytics, and interactive visualization of data for agricultural wholesale markets in developing nations. These individual pieces have been developed separately and show only particular functions or levels of the pipeline in action; this project offers a complete solution. Moreover, this project fills this void with an already working frontend that depicts the intended use of the entire system before further development of the Capstone 2 backend begins.

CHAPTER 3: System Analysis

This chapter provides a full system analysis of the Kalimati Vegetable Price Monitoring, Analysis, and Forecasting System from the role of Data Engineer and System Architect. The chapter will be divided into 3 sections: A theoretical analysis of the SWOT of the 7 core technologies of the stack, an technical analysis of the algorithms and libraries being utilized, and a specification of all of the functional and non-functional requirements. The following chapter throughout will use the term "Capstone 1 deliverable" when referring to a working React prototype, rendered by generated mock data, contrasted against the "Capstone 2 target architecture", which will describe the entire project including the Python ETL pipeline, PostgreSQL database, Flask REST API, and ARIMA forecasting module.

Initially the project was intended to cover both front-end and back-end development during both capstones together. Once clarified that Capstone 1 was simply meant to cover a mock-up or prototype, the working prototype which had already been developed ahead of schedule has been included as the Capstone 1 deliverable. The functional prototype is a much better deliverable than an static wireframe or Figma mockup, the prototype is dynamic, uses the mock data, and uses the same frontend structures that will eventually consume the data from the real backend in capstone 2; the only required changes during Capstone 2 integration will be the replacement of the mock-data generator with the real pipeline, and no modification on the front end will be needed.

3.1 Theoretical Analysis: SWOT

A SWOT analysis is performed on all seven of the main project stack technologies. Every SWOT is applied in the context of the specific demands of the Kalimati Price Monitoring system, not just evaluating a technology in the abstract. Claims are justified where necessary, and citations to the literature review used where possible.

3.1.1 Python

Python is the chosen language for all the backend and data engineering components: ETL process, Flask REST API, SQLAlchemy ORM specifications and ARIMA modeling. The following SWOT analysis assesses the benefits of using python for these components within the context of this project:

Table 3: Python SWOT Analysis

STRENGTHS	WEAKNESSES
• Mature ecosystem for data science, numerical calculation and time-series modelling - Pandas, NumPy, Statsmodels lack in native comparable (Go, Java) stacks. • Clear and concise syntax used to implement the ETL cleaning logic	• True multi-threaded parallelism is prevented by the GIL; CPU intensive ARIMA fitting cannot run simultaneously with API requests without multi-processing. • Slow I/O throughput as compared to Go or Rust (relevant if the Flask API

(forward fill, interpolation, anomaly detection). • Flask and SQLAlchemy both being python-native allow us to build a single-language backend stack, with no overhead for inter-language data serialisation. • The recent, although still experimental, enhancements to python's free-threaded mode (3.12+) along with the early stages of python JIT compiler will finally overcome the old bottleneck of the GIL. • Has already been independently validated by Xia (2024) in his implementation of ARIMA models on the Kalimati dataset where he utilized a stack composed of Pandas, NumPy and Statsmodels, exactly what is planned here.	handles significant concurrent load at production). • Linearly scaling memory footprint for in-memory pandas DataFrames; multi-year Kalimati data per commodity per request can potentially become a memory issue if not chunked. • Dependency management headaches: conflict between Pydantic v1 and v2, pinning of Statsmodels versions and NumPy API difference between 1.x and 2.x.
OPPORTUNITIES • Experimental free-threaded mode in Python 3.13 eliminates GIL enabling simultaneous ARIMA fitting and API request processing from a single process-direct improvement for Capstone 2 API response times. • Same Python pipeline that clean-loads data for ARIMA, can also be adapted for Capstone 2 to perform SARIMA or even Prophet model training with minimal architectural modifications (no language switch). • Polars, is a drop-in performance upgrade if Kalimati data size increases to the point where Pandas bottleneck becomes an issue. (A rust backed DataFrame library which offers a Pandas compatible API).	THREATS • Go and Rust are more commonly becoming used to develop high throughput API services, and they will begin to eat away at the case for python performance with the Flask API layer on subsequent iterations. • Python's problematic dependency management will silently break library version compatability in production if you update things such as your server OS without the library versions being updated and released to match. • Synchronous WSGI handling from flask makes it impossible to handle in a production environment without a Gunicorn worker configuration-a risk for misconfiguration in Capstone 2.

3.1.2 Pandas

The main library used for manipulating the data within the ETL pipeline is Pandas which allows for the use of the DataFrame abstraction that is essential in data ingestion, data cleaning, reindexing the time-

series, imputing missing data and calculation of analytical methods such as moving average or calculation of the Coefficient of Variation.

Table 4: Pandas SWOT Analysis

STRENGTHS • Native DatetimeIndex provides direct date based indexing, daily resampling, and rolling window operation-the 3 core operations of Kalimati's ETL process. • Forward_fill() and interpolate() implement exactly the tiered imputation method detailed in the pseudocode of data cleaning, using exactly one line per tier of imputation. • Groupby() and date resampling together make per-commodity time series into a single, vectorized, pass over the data, avoiding inefficient row-by-row loops. • Explicitly cited by McKinney (2010), the original library paper, and applied to agricultural ETL data cleaning preprocessing in Rana et al. (2024) confirming the method's appropriateness. • Readcsv() with parsedates readily parses Kalimati CSVs directly without needing any manual date-parsing code.	WEAKNESSES • In-memory only: A large (multi-year) Kalimati data file loaded as a single DataFrame may run out of RAM on a low spec server and will not run unless chunking is employed. • Pandas operations are single threaded and synchronous. When processing all commodities iteratively and serially (rather than in parallel), the ETL rate is restricted. • The N+1 risk: Running a per-row operation inside an iterative loop instead of using vectorized transforms can significantly reduce the speed of the cleaning pipeline.
OPPORTUNITIES • If data volumes increase from Capstone 2, the cleaning pipeline can be moved to Polars (a library which supports an identical API, but runs the underlying DataFrames through a multi-threaded Rust engine) without a significant rewrite. • Pandas DataFrames are the natural data type to feed into Statsmodels ARIMA class (i.e. There will be no need to serialise in between the ETL stage and the forecasting stage)	THREATS • The rising usage of polars as a performance-first option could quickly make pandas obsolete for big-data engineering work although the documentation/community support for it are second to none at the current scale. • An inefficient cleaning pipeline that uses non-vectorised functions over the multi-year kalimati dataset

The technical indicators supplied by the pandas-ta library (EMA, Bollinger Bands) will be readily available to augment the analytics stage in Capstone 2 without the need for an extra dependency.	could make ET L ingestion unfeasibly slow making Capstone 2 integration testing take a long time.
--	--

3.1.3 NumPy

NumPy forms the numerical computation underpinnings for vectorised array operations, standard deviations that are necessary for analysis of volatility and for detecting outliers, and indeed the array primitives upon which Pandas DataFrames depend.

Table 5: Numpy SWOT Analysis

STRENGTHS - Np.std() The population std, as used in CV volatility ranking (a primary feature). - Numpy Broadcasting The mechanism of element-wise operations without looping-as implemented in Z-score anomaly detection module to detect records > 2 SD away from the moving average. - Harris, E.M. Et al. (2020) Nature report onNumPybeing a corner stone array programming library to the whole Python scientific stack. It is well-maintained by the scientific computing community with the guarantee of long term existence. - Directly supports Pandas (Pandas Series are internally built on numpy arrays), and Statsmodels (model fitting is built on numpy matrix calculation in the background).	WEAKNESSES - The static typing of the array structure negates the convenience of Python's dynamic typing-one has to be familiar with rules for dtype promotion to prevent "silent" integer-float conversions that might produce errors. - The broadcasting rules may not be intuitive to non-scientific computing personnel within the team, and this might be another source of bugs for custom analysis.
OPPORTUNITIES numpy.linalg gives access to tools which can be used to carry out matrix factorisation and decomposition to perhaps further down the line be used	THREATS The GPU accelerated CuPy substitute for CPU-limited NumPy operations might in the future make NumPy look underpowered as scale increases,

to analyse commodity price correlation through PCA. • CuPy supplies a GPU accelerated API that closely mimics the functionality of numpy - which could be a future consideration, should it be appropriate to move the analytics engine onto a GPU enabled server for large data sets.	however, not an issue for Kalimati dataset size at present. • API breaking changes with the introduction of NumPy 2.0; libraries pinning to NumPy 1.x and libraries that require NumPy 2.x might raise a dependency resolution conflict in the environment.
---	--

3.1.4 Statsmodels

The function for time-series model fitting and forecasting in Statsmodels is the ARIMA class. The `adfuller()` function is available in Statsmodels and is used for testing for ADF stationarity. The SARIMAX class allows for seasonal model extensions and is another key function within Statsmodels which is the main analytic library in Capstone 2's forecasting module.

Table 6: Statsmodels SWOT Analysis

STRENGTHS	WEAKNESSES
• One class, <code>statsmodels.tsa.arima.model.ARIMA</code> provides a full workflow: specifying the model, fitting by MLE, diagnostic plots(ACF/PACF), forward forecasting, confidence interval for forecast, along with rich documentation. • <code>Adfuller()</code> provides a ready to use implementation for Augmented Dickey-Fuller test in which automated stationarity pipeline (ADF test differencing retest) described by the pseudocode can be built. • SARIMAX is available with no API change required as a direct extension, since <code>ForecastService</code> is just accepting the order tuple, hence an ARIMA SARIMA extension requires only one configuration setting.	• Choice of (p, d, q) for ARIMA requires domain knowledge and inspection of diagnostic plots (ACF/PACF graphs) as there is no automated way to select the optimal order as there is in Facebook Prophet. • The operation of fitting an ARIMA is computationally expensive in the context of large time-series data and the training time may become an issue for all the commodities at Kalimati and several years at the time of implementation in Capstone 2. • Currently the placeholder in Capstone 1 simply simulates a naive random-walk while the implementation of the Statsmodels ARIMA model is planned in Capstone 2..

It was confirmed with Xia (2024) and Udari and Hemachandra (2024) with vegetables price dataset whose features match the characteristic of Kalimati.	
OPPORTUNITIES - SARIMAX allows for exogenous regressors; future versions of Capstone could add variables such as the weather or festival calendar, which could act as external regressors, without modifying the model class itself. - The auto_arima function in the pmdarima package allows for automatic order selection as a Statsmodels-compliant wrapper, removing the need for manual parameter tuning in Capstone 2.	THREATS - Both Facebook Prophet and Darts have automatic parameter tuning, seasonal forecasting and also provide good documentation which would allow people who are not domain experts to understand easily – if these seem more accessible to the Analytics Lead, the ForecastService has an interface so they can be swapped out without altering the API. - Danger of models being overfitted to shorter commodity time-series (commodities with less than 365 clean records (post-imputation) would be too short for accurate ARIMA modelling).

3.1.5 Flask

We used Flask as a light-weight Python web framework for the REST API layer in Capstone 2. We provide an interface that is reachable through HTTP GET requests from the React front-end, returning commodities list, historical price data, analytical results, and forecast result through JSON.

Table 7: Flask SWOT Analysis

STRENGTHS - Low overhead and consciously simple; This project does not use an ORM, does not require a dedicated admin panel, does not have a template engine in the application layer, which remains merely the concern of routing and JSON serialisation. - Clean modular routing through Flask's blueprint architecture; it's simple to	WEAKNESSES - The flask has built-in support for WSGI (synchronous); with no multiple gunicorn worker processes being used a single flask process could only ever handle a single request, and hence could not compete in a production environment. - Automatic API documentation is not included (unlike FastAPI), this must either
--	--

organise routes under dedicated modules such as /prices, /analytics, and /forecast. These mirror the project's architecture, and it's easy to set up separate Flask blueprints to define them in each appropriate module. • CORS handling trivial from Flask-CORS; when developing it's straightforward to allow cross-origin requests from the React client using this extension. The browser's same-origin policy is just ignored with no complex server setup to worry about. • Simple Flask-SQLAlchemy Integration: Flask-SQLAlchemy's ease of integration is the biggest plus for this project. Since we already use SQL Alchemy for its database interaction this extension enables smooth and tight integration with Flask minimising boilerplate code.	be maintained manually or added with flask-smorest. • Lack of automatic validation, request parameter validation must be performed manually with Pydantic or Marshmallow as the framework doesn't perform it automatically.
OPPORTUNITIES • The React frontend was constructed using a static JSON structure; the simplicity of Flask's routing makes it possible to just pass back the same static structure for each API endpoint with no changes needed to the frontend. • Flask Blueprints enable each team member (Backend engineer, Analytics Lead, Data engineer) to work on their API sections separately, reducing merge conflicts.	THREATS • FastAPI is the new default standard for python REST APIs because of its support for async, auto docs and Pydantic-based validation, in the Capstone 2 code review, flask looks a bit old by comparison. • WSGI bottleneck: without a correctly configured Gunicorn + Nginx, concurrent requests from the dashboard will have terrible latency with any sort of load.

3.1.6 SQLAlchemy

PostgreSQL and the Python application are joined together by the ORM layer that SQLAlchemy provides. It outlines the relational database structure as Python classes, handles the database sessions and sends parameterised queries that load the ETL data and are also used to obtain data for the API.

Table 8: SQLAlchemy SWOT Analysis

STRENGTHS	WEAKNESSES
-----------	------------

• Database-agnostic ORM-This means you could move the entire system from developing on SQLite to production on PostgreSQL just by changing one string in the configuration-no route handlers, ETL functions, or schema classes need to be rewritten. • Schema-as-code-The Commodity and DailyPrice ORM models actually completely specify the schema and there is no schema-code drift as these models become the single source of truth for the whole team. • Parameterised queries are performed automatically through the ORM-there's no chance of SQL injection as we never write plain SQL strings in application code. • Fully async-compatible session management provided by SQLAlchemy 2.0. This gives a roadmap to moving to non-blocking database operations when Flask is eventually replaced with an async web framework.	• ORM abstraction is likely to create inefficient SQL queries for analytical aggregations (window functions, multi-commodity date joins, etc.), and performance sensitive analytical queries will need to be written in the SQLAlchemy Core Expression Language rather than via the ORM. • The ORM's propensity to allow the easy injection of N+1 queries (running N independent SELECT statements within a loop rather than a single joined query), is another significant performance antipattern within an API at analytical endpoints operating over many records. • The requirement for Alembic migration scripts for any schema change adds to operational complexity. The production schema will require manual and in order deployment.
OPPORTUNITIES • Schema migration is supported using Alembic and SQLAlchemy, tracking Capstone 1 to Capstone 2's schema evolution (and can also reverse it). • Using SQLAlchemy 2.x's Core Expression Language, a more low-level SQL query builder to tackle high-performance analytic queries without completely abandoning the ORM.	THREATS • SQLAlchemy - the author of FastAPI has released a unification of Pydantic schema classes and SQLAlchemy ORM models into one definition. A simpler solution that might even seem verbose for the dual-class SQLAlchemy + Pydantic combo we've been using up to now. • Misconfiguration of connection pooling could lead to connections exhausting for the analytics dashboard with multiple concurrent connections.

3.1.7 PostgreSQL

The production relation database is a PostgreSQL. It holds normalised commodity metadata, daily price entries and pre calculated analytical results. The zero-configuration replacement for development is a SQLite.

Table 9: PostgreSQL SWOT Analysis

STRENGTHS • Achieves Full ACID (Atomicity, Consistency, Isolation, Durability) compliance which guarantees not to load any incorrect or corrupted data into database while loading data in an ETL batch. • Native date/time functions (datetrunc, extract, generateseries) can help make queries faster while performing time range filter and time-series aggregate can be computed natively using SQL and is not required to be computed at application layer. • Native support of window functions (ROW_NUMBER, LAG, LEAD, rolling aggregates) would be used to move moving average pre-computation to completely reside in the database layer, shortening the time taken using Pandas. • TimescaleDB is an open-source extension of PostgreSQL that turns PostgreSQL into a hypertable optimized time-series database by using automatic partitioning based on time ranges which is well suited to theKalimati daily_prices table.	WEAKNESSES • Demands an available server process with authentication configured-unlike file-based zero-configuration SQLite-which requires more operational overhead to set up during development and deploy on the Capstone 2 project. • The table schema may change between development stages requiring manual execution of migrations through Alembic which represents a procedural hazard if version control is not diligently managed. • Using a cloud provider for managed PostgreSQL (e.g. AWS RDS, Google Cloud SQL) incurs persistent running costs for any production deployment beyond local or classroom servers.
OPPORTUNITIES • TimescaleDB extension augments PostgreSQL, making it into a time-series optimized database capable of handling our main query pattern of sequentially looking up daily prices at the analytics endpoints, without having to use a separate technology stack.	THREATS • Simplicity of SQLite may tempt to use in production as it would be easier to set up than configure PostgreSQL, which lack of write support would corrupt the data under even minimal concurrent ETL and API workload. • PostgreSQL major upgrades (i.e. 14->17) might require a data migration

- pgstatstatements extension gives query-level performance analysis, helping to locate slow analytical queries that may be a performance bottleneck to tune, and optimize before Capstone 2. - Pgpartman extension will perform date-range partitioning on the dailyprices table automatically, increasing query performance as the dataset grows over many years of data.	plan, as the project server would not be compatible with auto upgrade.
---	---

3.2 Technical Analysis

This section discusses the algorithmic and computational approach for the system's essential data engineering and analysis functionalities. The computation cost, input assumptions, and predicted behavior of each technique using Kalimati market data is discussed.

3.2.1 Data Pipeline Architecture

The DataPipeline class runs a four stage ETL process. 1: Data extraction from the CSV file and date parsing. 2: Data transformation: normalize names, drop duplicates based on the unique tuple (commodity, date), reindex by date, handle missing values (imputation). 3: Data validation against data integrity constraints. 4: Data loading: use SQLAlchemy for batched database insertion. The pipeline is idempotent in nature, meaning it will not insert duplicate data into the database, even if run multiple times on the same data set, as exact duplicate values (on commodity, date) will be dropped during the cleaning process and the database enforces uniqueness of combination of commodity and date via schema constraints.

The tiered missing value imputation is a technically important part of the pipeline. When a value is missing between one and three days it is forward-filled; in a market context the value is unlikely to change much over such a short period that could result from weekend or public holiday closures. Between 4 and 14 days the value is linearly interpolated from the nearest two available data points, making the assumption that it changes more smoothly over such gaps compared to a step change. Values more than 14 days old are left with a UNRESOLVABLE value in the database, set to NULL, because linear interpolation beyond such periods is likely to create artificial trends in the data, which are contrary to the dynamics of market price behavior. This imputation strategy is in line with the methodologically advised techniques of Rahm and Do (2000) and Kazijevs and Samad (2023) as discussed in chapter 2.

3.2.2 Analytical Methods

The Moving Average(7-day & 30-day SMA) is the most basic and easily interpretable of the trend indicators. It averages the previous N prices in order to create a smoothed series to remove daily fluctuation while maintaining the directional trend. Mathematically calculated as:`Pandas.rolling(window=N).mean()`, moving averages are computed in ETL and stored in the `analytics_cache` table to avoid recomputing each time the API is called.

The Coefficient of Variation($CV = \text{standard deviation} / \text{mean}$) is computed in order to provide a comparative ranking of the volatilities of the commodities against each other. Raw standard deviation does not provide this ranking since commodities can have a very different scale for prices. For example the price of Garlic is expected to be \$200/kg, whereas spinach should be ~\$3/kg. If these were graphed the std dev of Garlic prices could easily exceed \$10 while spinach std dev is very small in absolute terms; therefore the CV gives us a relative scale for the volatilities of these two different items.

The Z-score anomaly detection algorithm flags points as an anomaly if the data point lies more than 2 standard deviations away from the 30 day moving average, and points that are flagged will be marked in the trend chart. Two standard deviations represents approximately 95% of all non-anomalous normally distributed points, therefore providing a reasonable cutoff to denote significant deviations in price as opposed to normal daily fluctuations.

Within the automatic ARIMA forecasting pipeline, there's an automatic Augmented Dickey-Fuller test on whether the series is stationary or not. If the p-value is greater than 0.05 (series is non-stationary), the series is then differenced once and tested again. If it is not stationary after 1 difference, it is difference again until the series is stationary. Default parameters of (2,d,1) is set from Udari & Hemachandra (2024) who determined (2,1,1) to be the optimal model for vegetable prices similar to the commodities here. Performance will be analyzed over a 30 day hold-out data set for MAE, RMSE, and MAPE metrics shown on the forecast tab.

3.2.3 Mock Data Generation Strategy (Capstone 1 Prototype)

Instead of the actual ETL pipeline and the ARIMA module, the Capstone 1 prototype employs a class named `MockDataGenerator`. This class uses a random-walk with mean reversion algorithm. It generates the daily historical prices of 8 commodities over 180 days. Each commodity is first assigned with a reasonable initial historical price. At each step it gains a Gaussian random shock (multiplied by a volatility coefficient specific to each commodity), and it is affected by a mean-reversion mechanism to restore the price to the initial level. Random shocks resulting in unexpected spikes are generated by giving a 5% chance each day of jumping the price by 20% immediately. Unsurprisingly, these time series exhibit a short-term noise component, periodic spikes and slow drifts without the Kalimati data. This output is structured precisely as the actual data pipeline, which means it is a dictionary, keyed by the commodity name. Each commodity's value in this dictionary is a nested dictionary with array values representing: date, price, the 7-day and 30-day MAs, and the forecast dict. Since this output structure is unchanged, the React frontend would not require any modification.

3.3 Requirement Specifications

3.3.1 Functional Requirements

Table 10: Functional Requirements

ID	Feature	Description	Phase	Owner
FR-01	Data Ingestion	The system will retrieve historical Kalimati prices from the ErKiran/kalimati GitHub CSV repository and insert them into an organized SQL database.	Cap 2	Data Engineer
FR-02	Data Cleaning	The system will implement a layered imputation method for missing data (forward-fill for 3 days and linear interpolation for 4-14 days, normalize commodity names, and date-price data validation.	Cap 2	Data Engineer
FR-03	Trend Analysis	The system will calculate and present 7 and 30-day moving averages for any chosen commodity within a specified date range.	Cap 1+2	Analytics Lead
FR-04	Volatility Analysis	The system will calculate and rank the coefficient of variation for each commodity in order to identify which vegetables are the most price volatile.	Cap 1+2	Analytics Lead
FR-05	Price Forecasting	The system will predict a 30-day forecast of prices based on an ARIMA model and ADF stationarity testing. MAE, RMSE and MAPE will be calculated to assess prediction accuracy.	Cap 2	Analytics Lead
FR-06	Interactive Dashboard	A web interface will be designed containing commodity selection, date-range picker and three separate	Cap 1	Frontend Dev

ID	Feature	Description	Phase	Owner
		views (trend, volatility and forecast).		
FR-07	Prototype Deliverable	The final product for Capstone 1 will be a functional React prototype generated programmatically with mock data. It will display all three fundamental system views.	Cap 1	Data Engineer / Frontend
FR-08	Data Storage	Price records will be stored in a normalized relational SQL scheme containing three tables (Commodities, Daily Price, pre-calculated analytic outputs)	Cap 2	Database Dev
FR-09	API Layer	RESTful API calls will allow for easy interaction with commodity list, historical prices, calculated analytical outputs and forecast results from the backend.	Cap 2	Backend Engineer
FR-10	Anomaly Display	Data points will be displayed as anomalies if they fall outside two standard deviations from the rolling mean, clearly marked on trend line charts.	Cap 1+2	Frontend Dev

3.3.2 Non-Functional Requirements

Table 11: Non-functional Requirements

ID	Category	Description
NFR-01	Performance	Dashbard shall load and display charts in 3 secs for upto 2 yrs of price on a standard broadband.
NFR-02	Scalability	SQL Schema & ETL pipeline shall allow for many yrs of commodity data (min 10 yrs of daily) with no changes in schema/ ETL.

ID	Category	Description
NFR-03	Usability	UI requires no technical knowledge, commodity selection, date filtering and switching between charts can be achieved within 3 clicks from home page.
NFR-04	Maintainability	Shall have modular design, any layer (ingestion, processing, storage, analytics, frontend) should be upgradeable without affecting other layers.
NFR-05	Reliability	ETL Shall Log errors/ skipped rows on insertion for manual inspection and shall not fail.
NFR-06	Portability	Should run on any OS with python 3.9+, node.js 18+, PostgreSQL 14+ (no OS specific libraries).
NFR-07	Data Integrity	All prices should be normalized by name, validated for format and be ≥ 0 .
NFR-08	Accuracy	ARIMA Module shall achieve 15% MAPE for 30 days out-of-sample validation for stable commodities.

3.4 Summary of System Analysis

This Chapter outlines the systems analysis for the Kalimati vegetable price monitoring project in the context of a data engineer. The chapter begins with a quick look at the pros and cons of the seven tools chosen for the system in question: Python (a popular choice that is simple to learn, but can be slow with large data), Pandas (a simple tool for the cleaning and organizing of price data, but is very memory intensive), NumPy (a tool that performs computations rapidly, but is difficult for beginners), Statsmodels (a useful tool for performing ARIMA forecasting, but may not be the fastest), Flask (a light, simple framework for creating web APIs, but not suited for high-traffic websites), SQLAlchemy (a tool to easily and safely connect to a database from python; if poorly written it can create very slow queries), and PostgreSQL (a robust, secure, time-series compatible database that is a bit more difficult to set up compared to simply using flat files). This chapter then goes on to define the structure of the system, detailing that the data pipeline will take raw CSV files from Kalimati, cleanse them, correct missing prices by using a forward fill or interpolation methodology, then upload the data to a database. It goes on to detail which analytical measures will be performed; seven day and thirty day moving averages will be calculated to establish trends, a coefficient of variation will be calculated for volatility ranking, and 30 day price forecasts will be calculated via an ARIMA model. In order to ensure the Capstone 1 system can be functional, synthetic, but realistic price data is created using a random walk method so that the frontend system may properly generate charts. The functional and non-functional requirements are then defined;

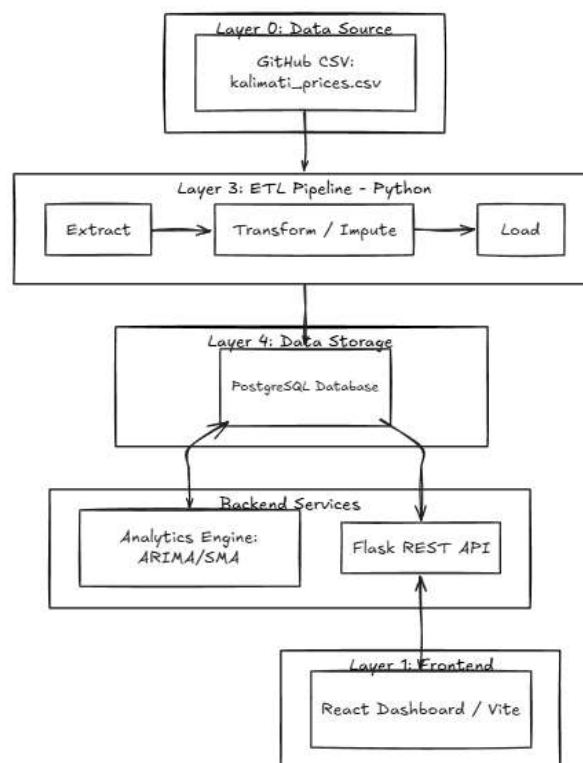
there are ten functional requirements that detail what the system must perform (collect data, clean data, provide trend charts, produce forecasts, interactive dash-board), there are eight non-functional requirements detailing the quality of the system (speed, usability, data reliability, forecasting accuracy up to 15% error), and it is stated that this chapter confirms the chosen tools are the best fit for the project at hand and accurately defines what the system will accomplish in Capstone 2.

CHAPTER 4: System Design

This chapter introduces the system design for the Kalimati Vegetable Price Monitoring, Analysis, and Forecasting System fully in the role of the Data Engineer and System Architect. Throughout the Capstone design process, an important design difference is enforced: a full frontend prototype developed in React, running against mock data, is the Capstone 1 deliverable, and the entire five-layer system, including a Python ETL pipeline, PostgreSQL database, Flask REST API, and an ARIMA forecasting model, is the final goal system architecture of Capstone 2. Diagrams show the entire target system design with special indication where features belong only to the Capstone 1 system design or where they belong to the target Capstone 2 system design.

4.1 System Architecture Diagram

The five-layer modular system's structure is designed as follows: One clear responsibility for each layer, communicated with neighbours through clear interface; Layer 1 and 2 communicating through REST API calls; Layer 2 to Layer 5 communicating through SQLAlchemy ORM queries. It is similar with DAKIS modular system we investigated on Chapter 2 (Mouratiadou et al. 2023), in which it was clearly shown that layered architecture is used for an agricultural information system to interconnect data, analytic, and user interface.



System Architecture Diagram

Figure 1: System Architecture Diagram

The Layer 0 split has major architectural consequences: the Mock Data Generator (Capstone 1) and the actual Kalimati GitHub CSV (Capstone 2) output the same format- date indexed price arrays indexed by commodity. All five of the layers above Layer 0 are architecturally identical in Capstone 1 and Capstone 2. The only thing that changes upon integration in Capstone 2 is the data source-the code used for the front end and analysis does not change.

4.2 Data Flow Diagram

Data Flow Diagram - The data flow diagram shows the path from the initial raw Kalimati CSV to the display of an interactive chart data in the front end. This shows separately data flow of capstone 1(mock generator React) and capstone 2(CSV ETL PostgreSQL Flask React)

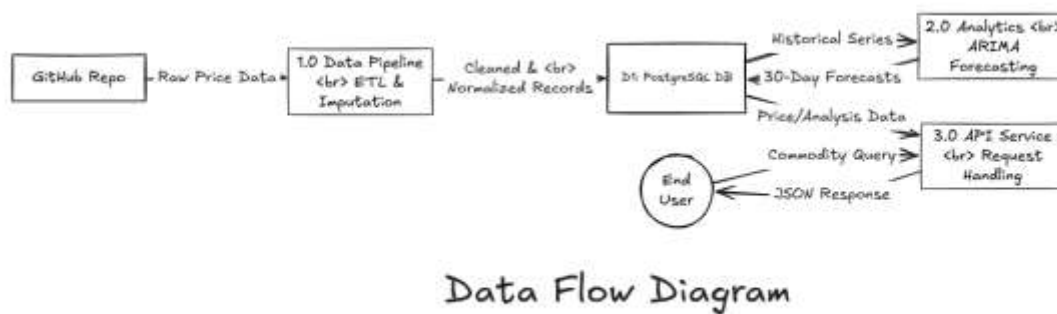
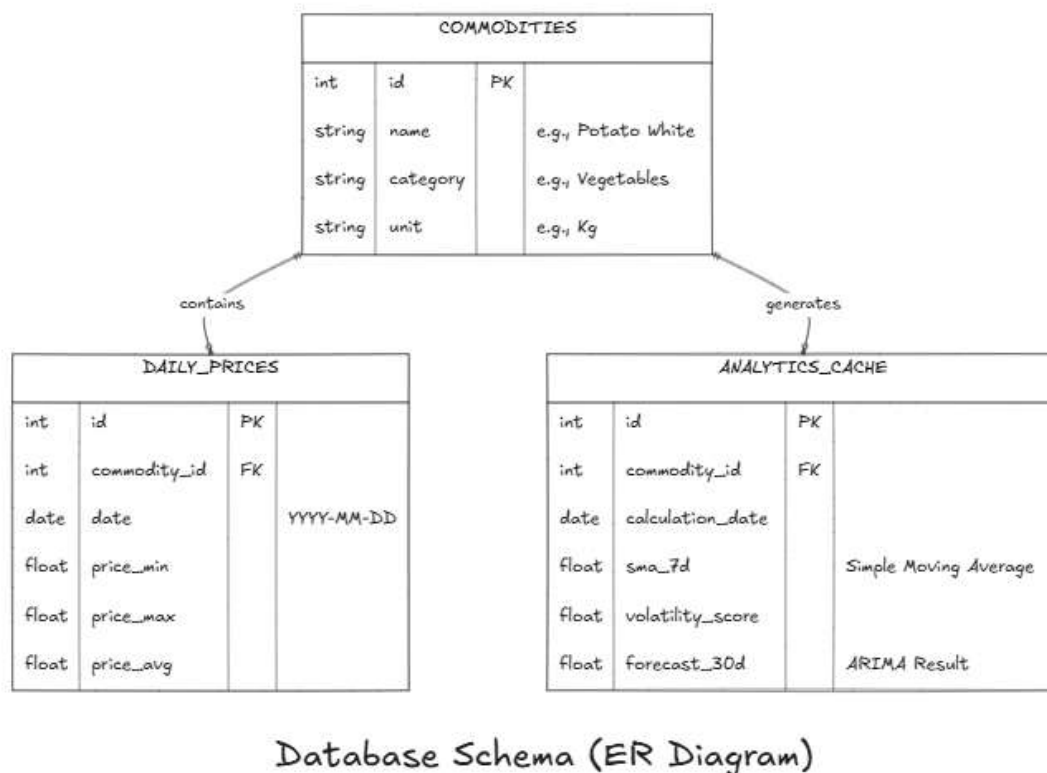


Figure 2: Data Flow Diagram

4.3 Database Schema Diagram

The relational schema adheres to the normalisation principles outlined by Akadal and Satman (2022). Commodity metadata is separated from price records in order to prevent redundancy and facilitate per-commodity and cross-commodity querying. The analytics_cache table holds rolling statistics and forecasts calculated beforehand, in order to separate the potentially costly computation process from that of serving an API request.



Database Schema (ER Diagram)

Figure 3: Database Schema Diagram

The UNIQUE constraint on (commodityid, recorddate) in DAILYPRICES is applied at the database level and ensures that for any one commodity, no more than one price record will be inserted per day no matter how many times the ETL process is run and over. The analyticscache table is then written to by the AnalyticsEngine, which runs in an entirely separate, schedulable process and is not tied to the Flask API, ensuring that costly ARIMA refitting and volatility re-calculation does not impact the speed with which real-time dashboards can be requested.

4.4 Use Case Diagram

There are two main actor groups which interact with the system: End Users (consumers, owners of small enterprises and market analysts) interact through the dashboard and System Operators (the 5 team members) interact through backend levels in the development. Use cases UC-01 to UC-05 are all implementable in the Capstone 1 prototype. Use cases UC-06 to UC-10 will be postponed for Capstone 2.

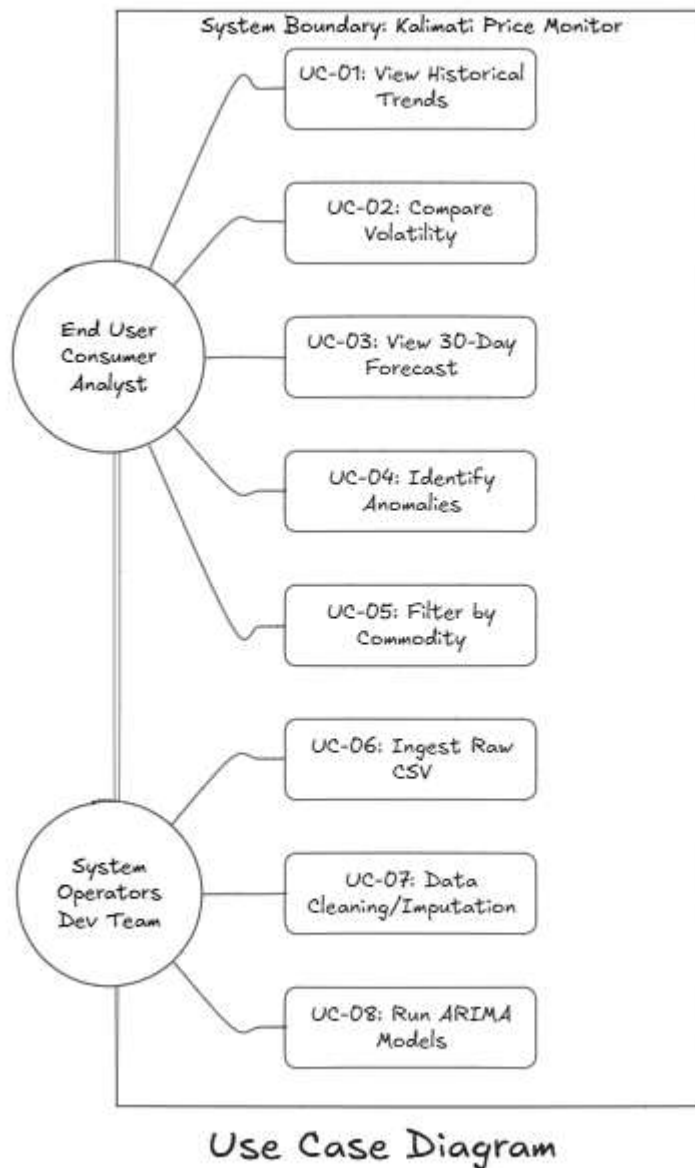
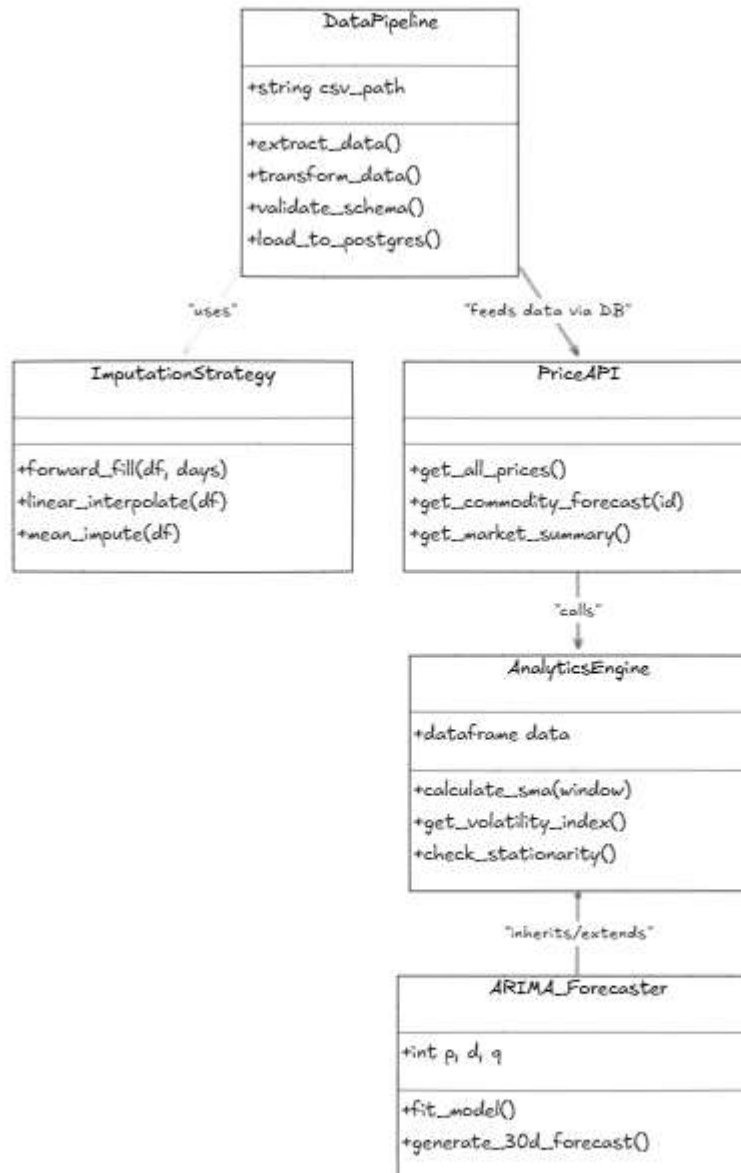


Figure 4: Use Case Diagram

4.5 Class Diagram

The objects of the system comprise 5 main classes. Commodity and DailyPrice are ORM-mapped database entity classes. DataPipeline encapsulates the complete ETL flow. AnalyticsEngine encapsulates all computations for analysis and forecasting. MockDataGenerator replaces DataPipeline and real source in Capstone 1.

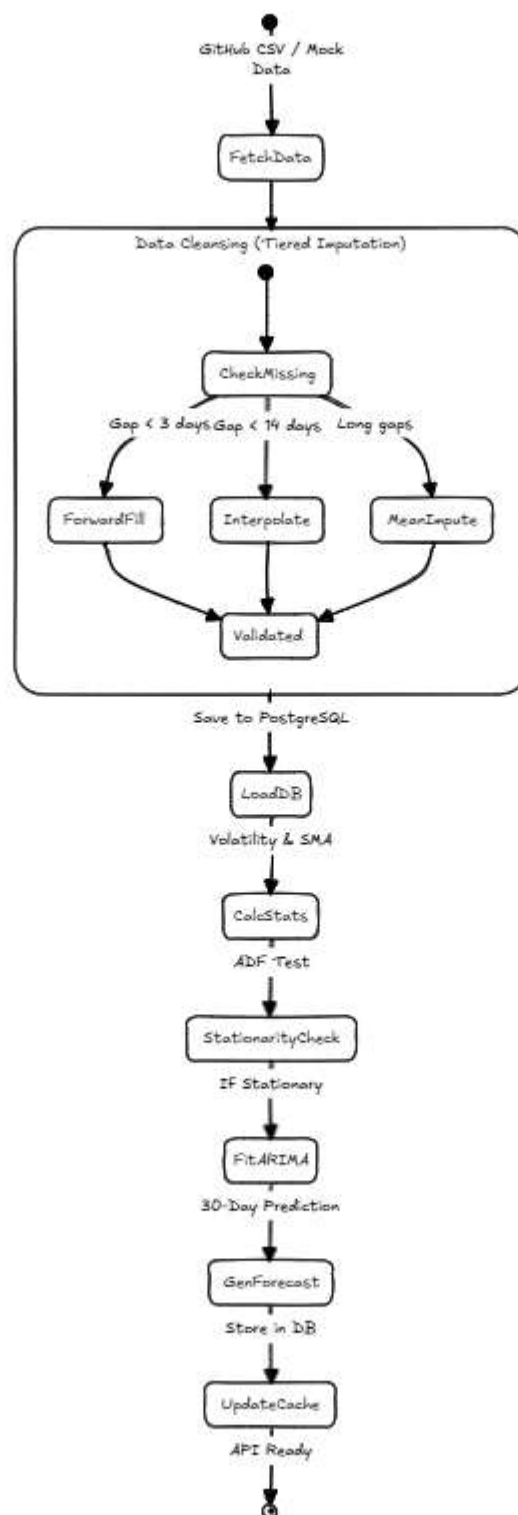


Class Diagram

Figure 5: Class Diagram

4.6 Activity Diagram

This activity diagram depicts the user's entire price trend and forecast request life-cycle: from triggering an HTTP request through data retrieval, analytics calculation, to sending a JSON response. This activity diagram displays the Capstone 2 full system flow, while the Capstone 1 mock path is a dashed alternative entrance to the diagram in Layer 0.

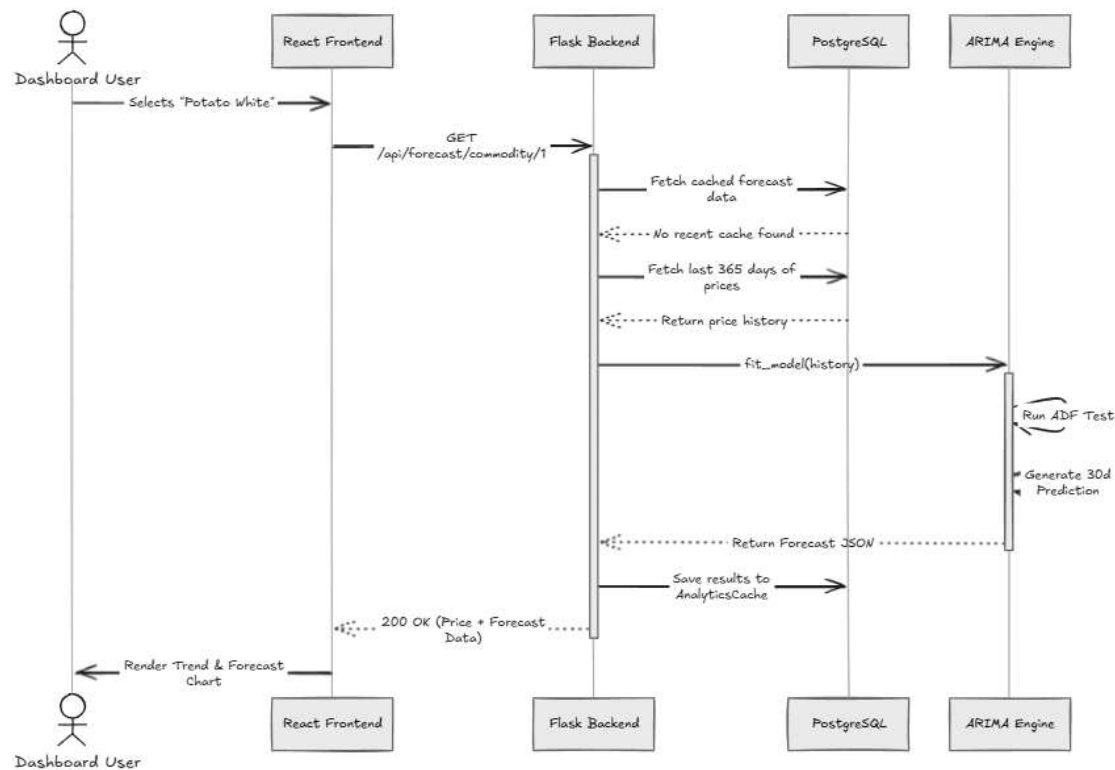


Activity Diagram

Figure 6: Activity Diagram

4.7 Sequence Diagram

This sequence diagram shows how system elements collaborate for a price forecast request; which is the most sophisticated analytical process in the system as it involves ETL load data, ADF test of stationarity, ARIMA fitting, confidence interval generation.



Sequence Diagram

Figure 7: Sequence Diagram

4.8 Pseudocode

4.8.1 ETL Pipeline – DataPipeline.run()

Execution of the complete ETL workflow. Including ingestion, normalization of commodity names, date-range reindexing, tiered missing value imputation, price validation and batched insertion into the database. This is the main deliverable of Capstone 2 under the direct supervision of the Data Engineer.

```

CLASS DataPipeline:
  ATTRIBUTES:
    source_path  — str: path to xalimati CSV file or URL
    db_session   — SQLAlchemy Session object
    log          — list[str]: record of all skipped/flagged rows

  FUNCTION load_csv():
    df ← pandas.read_csv(source_path, parse_dates=['date'])
    IF df is empty OR 'date' column missing THEN
      RAISE FileParseError — HALT pipeline
    RETURN df sorted ascending by date

  FUNCTION clean(df):
    // Step 1: normalize commodity name variants via alias dictionary
    df['commodity'] ← apply name_map(df['commodity'])
    // Step 2: drop exact duplicate (commodity, date) pairs
    df ← df.drop_duplicates(subset=['commodity', 'date'], keep='last')
    // Step 3: reindex each commodity to full daily date range
    FOR EACH commodity IN df.commodity.unique():
      sub ← df[df.commodity == commodity].set_index('date')
      full_range ← date_range(sub.index.min(), sub.index.max(), freq='D')
      sub ← sub.reindex(full_range)
      gap_sizes ← compute_consecutive_null_lengths(sub['avg_price'])
      FOR EACH gap IN gap_sizes:
        IF gap.length ≤ 3: apply forward_fill(sub)
        ELIF gap.length ≤ 14: apply linear_interpolation(sub)
        ELSE: log('UNRESOLVABLE GAP: ' + commodity + ' from ' + gap.start)
    RETURN merged df

  FUNCTION validate(df):
    invalid ← rows WHERE min_price < 0
                      OR max_price < min_price
                      OR avg_price IS NULL
    log('SKIPPED ' + count(invalid) + ' invalid records')
    RETURN df EXCLUDING invalid rows

  FUNCTION load_to_db(df):
    FOR EACH row IN df:
      commodity_id ← db_session.query(Commodity).filter_by(name=row.commodity).id
      IF commodity_id IS NULL: INSERT new Commodity first
      INSERT DailyPrice(commodity_id, row.date, row.min_price,
                        row.max_price, row.avg_price)
    db_session.commit()

  FUNCTION run():
    df ← load_csv()
    df ← clean(df)
    df ← validate(df)
    load_to_db(df)
    log('PIPELINE COMPLETE — loaded: ' + count(df) + ' records')

```

4.8.2 ARIMA Forecasting – AnalyticsEngine.forecast()

```

FUNCTION forecast(commodity_id, steps=30):
  records ← DailyPrice.get_prices(commodity_id, start, end)
  series ← pandas.Series(records['avg_price'], index=records['date'])

  // Step 1: ADF stationarity test
  p_value ← statsmodels.adfuller(series)[1]
  d ← 0
  IF p_value > 0.05:
    series ← series.diff().dropna(); d ← 1
    IF statsmodels.adfuller(series)[1] > 0.05:
      series ← series.diff().dropna(); d ← 2

  // Step 2: Fit ARIMA(2, d, 1) -- baseline from Udari & Hemachandra (2024)
  fitted ← statsmodels.ARIMA(series, order=(2, d, 1)).fit()

  // Step 3: 30-day forecast with 95% confidence intervals
  forecast_values ← fitted.forecast(steps=steps)
  conf_intervals ← fitted.get_forecast(steps).conf_int(alpha=0.05)

  // Step 4: Holdout accuracy on last 30 records
  actuals ← series[-30:]
  predictions ← fitted.predict(start=len(series)-30, end=len(series)-1)
  MAE ← mean(abs(actuals - predictions))
  RMSE ← sqrt(mean((actuals - predictions)^2))
  MAPE ← mean(abs(actuals - predictions) / actuals) * 100

  RETURN {forecast: forecast_values, conf_intervals: conf_intervals,
          MAE: MAE, RMSE: RMSE, MAPE: MAPE, order: (2, d, 1)}

```

4.8.3 Mock Data Generator – Capstone 1 Prototype

```

CLASS MockDataGenerator:
  ATTRIBUTES:
    commodities ← ['Tomato','Potato','Onion','Cauliflower',
                  'Cabbage','Spinach','Broccoli','Garlic']
    base_prices ← {Tomato:60, Potato:35, Onion:50, Cauliflower:80, ... }
    volatility ← {Tomato:0.08, Potato:0.04, Onion:0.07, ... }
    reversion_speed ← 0.05
    days ← 180

  FUNCTION generate_prices():
    FOR EACH commodity IN commodities:
      prices ← [base_prices[commodity]]
      FOR day ← 1 TO days:
        prev ← prices[day - 1]
        reversion ← reversion_speed * (base_prices[commodity] - prev)
        shock ← random.normal(0, volatility[commodity] * prev)
        IF random.uniform(0,1) < 0.05:
          shock ← shock + random.choice([-0.20, +0.20]) * prev
        prices.append(max(5.0, round(prev + reversion + shock, 2)))
      result[commodity] ← {
        dates: date_array, prices: prices,
        ma7: moving_avg(prices, 7), ma30: moving_avg(prices, 30),
        forecast: linear_forecast(prices, steps=30)
      }
    RETURN result
  // NOTE: This entire class is replaced by DataPipeline in Capstone 2

```

4.9 Summary of System Design

Chapter 4 describes the whole system design of the Kalimati vegetable price monitoring project from the perspective of a data engineer and system architect. It describes a tidy 5 layer modular system where Layer 0 is the data input (fake mock data for Capstone 1, real Kalimati CSV files for Capstone 2), then the data pipeline that cleanse and loads prices, then the PostgreSQL data base where prices are kept secure, then the analytical layer where trends, volatility and forecasts are generated, then the Flask API that ties all layers together, then the React user interface layer where all interactive charts are displayed. There are seven concise diagrams: the general system architecture, the system data flow, the database schema and structure (using 3 normalised tables: Commodities, Daily Prices and Analytics Cache), the use case diagram between users and system administrators, class diagram of the key Python class structure, the activity diagram showing what happens when the user inputs a request and sequence diagram showing the forecasting process. Simple pseudocode for 3 of the key processes is included: data ETL pipeline process, ARIMA forecasting engine operation and mock data generation process to ensure all prices generated would be realistically distributed. Overall this chapter demonstrated that system is structured, modular and well designed to be implemented with actual code for Capstone 2. The whole system design

allows for the frontend prototype (to be built during Capstone 1) to function in exactly the same way the system will behave during actual implementation.

References

- Akadal, E., & Satman, M. H. (2022). A novel automatic relational database normalisation method. *Acta Informatica Pragensia*, 11(3), 293–308. <https://doi.org/10.18267/j.aip.193>
- Bao, W., Su, W., Zhao, X., & Zhuang, J. (2025). A study on short-term vegetable price prediction based on the CNN-LSTM-Attention model. *Discover Food*, 5(1). <https://doi.org/10.1007/s44187-025-00426-2>
- Choong, K. Y., Raof, R. A. A., Sudin, S., & Ong, R. J. (2021). Time series analysis for vegetable price forecasting in e-commerce platform: A review. *Journal of Physics: Conference Series*, 1878(1), 012071. <https://doi.org/10.1088/1742-6596/1878/1/012071>
- Dinesh Kumar, P. (2025). Forecasting future vegetable prices using data science with Python. *International Journal of Research Publication and Reviews*. <https://ijrpr.com/uploads/V6ISSUE11/IJRPR55173.pdf>
- Giri, A., & Giri, V. R. (2024). Forecasting cauliflower prices in Nepal: A comparative analysis using seasonal time series and nonlinear models. *Cogent Food & Agriculture*, 10(1). <https://doi.org/10.1080/23311932.2024.2340155>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kazijevs, M., & Samad, M. D. (2023). Deep imputation of missing values in time series health data: A review with benchmarking. *Journal of Biomedical Informatics*, 144, 104440. <https://doi.org/10.1016/j.jbi.2023.104440>
- McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). SciPy. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Mouratiadou, I., et al. (2023). The Digital Agricultural Knowledge and Information System (DAKIS): Employing digitalisation to encourage diversified and multifunctional agricultural systems. *Environmental Science and Ecotechnology*, 16, 100274. <https://doi.org/10.1016/j.es.2023.100274>
- Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. *IEEE Data Engineering Bulletin*, 23(4), 3–13.
- Ramesh, D., & Khosla, E. (2019). Forecasting price volatility of fruits and vegetables using seasonal ARIMA. In *2019 International Conference on Data Science and Communication (IconDSC)* (pp. 1–5). IEEE. <https://doi.org/10.1109/IconDSC.2019.8816975>
- Rana, H., Farooq, M. U., Kazi, A. K., Baig, M. A., & Akhtar, M. A. (2024). Prediction of agricultural commodity prices using big data framework. *Engineering, Technology & Applied Science Research*, 14(1), 12652–12658. <https://doi.org/10.48084/etasr.6468>
- RECPHEC. (2016). A report on the study of vegetable markets in context of Kathmandu Metropolitan City. https://healthbridge.ca/dist/library/Study_of_Vegetable_Markets_in_Kathmandu.pdf

- Subhasree, C., & Arun Priya, C. (2016). Forecasting vegetable price using time series data. *International Journal of Advanced Research*, 4(11), 888–893.
- Udari, U., & Hemachandra, S. (2024). Price behaviour, commercialisation opportunities and forecasting future prices of selected vegetables in Sri Lanka. *Journal of Technology and Value Addition*, 6(1), 31–50.
- Vassiliadis, P. (2009). A survey of extract–transform–load technology. *International Journal of Data Warehousing and Mining*, 5(3), 1–27. <https://doi.org/10.4018/jdwm.2009070101>
- Xia, R. (2024). Comparative analysis of ARIMA and LSTM models for agricultural product price forecasting. *HSET Conference Proceedings*. <https://drpress.org/ojs/index.php/HSET/article/view/18553>
- Zhang, Q., Yang, W., Zhao, A., Wang, X., Wang, Z., & Zhang, L. (2024). Short-term forecasting of vegetable prices based on LSTM model — evidence from Beijing's vegetable data. *PLOS ONE*, 19(7), e0304881. <https://doi.org/10.1371/journal.pone.0304881>
- Zhao, C. (2024). A review of agricultural product price forecasting research. *BIO Web of Conferences*, 142, 01002. <https://doi.org/10.1051/bioconf/202414201002>
- Zhao, C., Wang, X., et al. (2025). A vegetable-price forecasting method based on mixture of experts. *Agriculture*, 15(2), 162. <https://doi.org/10.3390/agriculture15020162>

Appendix

Appendix A: Project Gantt Chart

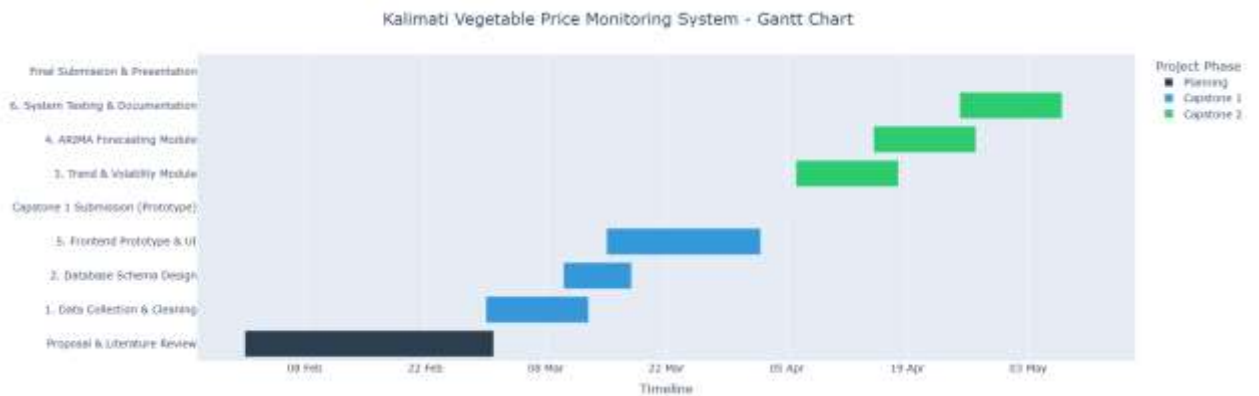


Figure 8: Gantt Chart

Appendix B: Excalidraw Link:

<https://excalidraw.com/#json=9BuNXc4Lk5bE36CiBmn-V,m5OO7AP7Jlttox5jHu4RqlA>

Appendix C: Meeting Minutes

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
7. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Subrna Thakurathi	Group: 24	Date: 29th March 2026	Meeting No: 3
--	------------------	------------------------------	----------------------

Project Title: A Web-Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market

Supervisor's Name: Saroj Regmi	Supervisor's Signature:
---------------------------------------	--------------------------------

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Individual Progress Report
- Review of Chapter 3 (System Analysis) and Chapter 4 (System Design) draft content
- Overall project milestones and timeline

Record of Discussion: (Noted by Student During Supervisory meeting)

- Supervisor checked up on the progress on each individual's contribution.
- Presented the completed Chapter 3 SWOT analysis of the seven core technologies (Python, Pandas, NumPy, Statsmodels, Flask, SQLAlchemy, PostgreSQL) and received feedback.
- Supervisor reviewed the system architecture diagrams (Architecture, Data Flow, Database Schema, Use Case, Class, Activity, Sequence) and provided corrections and guidance to improve the report.
- Supervisor reviewed the documentation report and advised on improving structure, depth, and clarity across all sections.

Action List (Noted by Student Next Supervisory meeting)

- Incorporate supervisor feedback into Chapter 3 and Chapter 4; refine the SWOT analysis sections and all system design diagrams.
- Improve and finalize the overall documentation report.
- Finalize the individual report.

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
7. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Subrna Thakurathi	Group: 24	Date: 1st April 2026	Meeting No: 4
---------------------------------	-----------	----------------------	---------------

Project Title: A Web-Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market

Supervisor's Name: Saroj Regmi	Supervisor's Signature:
--------------------------------	-------------------------

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Individual Progress Report
- Final guidance and improvements on system design documentation, data pipeline pseudocode, and literary review sections
- Report Finalization

Record of Discussion: (Noted by Student During Supervisory meeting)

- Supervisor checked up on the progress on each individual's contribution.
- Log sheets checking.
- Comments on the documentation — supervisor provided final remarks on the ETL pipeline design, database schema, and ARIMA forecasting pseudocode sections.

Action List (Noted by Student Next Supervisory meeting)

- Incorporate all final supervisor comments; ensure all pseudocode (DataPipeline, AnalyticsEngine, MockDataGenerator) and diagrams are finalized and consistent.
- Get the entire report checked by the supervisor and obtain the Turnitin report.
- Report Submission.

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
7. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name:
Subrina Thakurathi

Group: 24

Date: 22nd March
2026

Meeting No: 2

Project Title: A Web Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market

Supervisor's Name: Saroj Regmi

Supervisor's Signature:

Item for Discussion: (Noted by Student **Before** Supervisory meeting)

- Progress review on Chapter 1 and Chapter 2 drafts
- System Architecture design and component breakdown
- Team coordination and individual task updates

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Reviewed the completed chapter 1 and chapter 2 drafts
- Presented the proposed system architecture covering data pipeline, SQL database, React frontend and ARIMA forecasting module, supervisor approved the overall design direction
- Team tasks updates discusses: each member confirmed progress on their respective components

Action List (Noted by Student **Next** Supervisory meeting)

- Incorporate supervisor feedback into chapter 2, start working on Chapter 3
- Finalize the React frontend prototype with all three views (trend analysis, volatility, forecasting) using programmatically generated mock data.
- Complete the system architecture section in the individual report, including a component diagram and data flow diagram.
- Finalize the mock data generation script and integrate it into the React frontend prototype.
- Coordinate with the backend engineer (Robinson Ghimire) on API contract design for Capstone 2 integration.

Notes on use of the project log sheet:

8. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
9. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
10. A log sheet is to be brought by the STUDENT to each supervisory session.
11. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
12. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
13. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
14. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name:
Subrna Thakurathi

Group: 24

Date: 22nd March
2026

Meeting No: 2

Project Title: A Web-Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market

Supervisor's Name: Rukesh Shrestha

Supervisor's Signature: 

Item for Discussion: (Noted by Student **Before** Supervisory meeting)

- Progress review on Chapter 1 and Chapter 2 drafts
- System Architecture design and component breakdown
- Team coordination and individual task updates

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Reviewed the completed chapter 1 and chapter 2 drafts
- Presented the proposed system architecture covering data pipeline, SQL database, React frontend and ARIMA forecasting module, supervisor approved the overall design direction
- Team tasks updates discusses: each member confirmed progress on their respective components

Action List (Noted by Student **Next** Supervisory meeting)

- Incorporate supervisor feedback into chapter 2, start working on Chapter 3
- Finalize the React frontend prototype with all three views (trend analysis, volatility, forecasting) using programmatically generated mock data.
- Complete the system architecture section in the individual report, including a component diagram and data flow diagram.
- Finalize the mock data generation script and integrate it into the React frontend prototype.
- Coordinate with the backend engineer (Robinson Ghimire) on API contract design for Capstone 2 integration.

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the **STUDENT** to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
7. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Subrna Thakurathi	Group: 24	Date: 29th March 2026	Meeting No: 3
--	------------------	------------------------------	----------------------

Project Title: A Web-Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market

Supervisor's Name: Rukesh Shrestha

Supervisor's Signature: 

Item for Discussion: (Noted by Student **Before** Supervisory meeting)

- Individual Progress Report
- Review of Chapter 3 (System Analysis) and Chapter 4 (System Design) draft content
- Overall project milestones and timeline

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Supervisor checked up on the progress on each individual's contribution.
- Presented the completed Chapter 3 SWOT analysis of the seven core technologies (Python, Pandas, NumPy, Statsmodels, Flask, SQLAlchemy, PostgreSQL) and received feedback.
- Supervisor reviewed the system architecture diagrams (Architecture, Data Flow, Database Schema, Use Case, Class, Activity, Sequence) and provided corrections and guidance to improve the report.
- Supervisor reviewed the documentation report and advised on improving structure, depth, and clarity across all sections.

Action List (Noted by Student **Next** Supervisory meeting)

- Incorporate supervisor feedback into Chapter 3 and Chapter 4; refine the SWOT analysis sections and all system design diagrams.
- Improve and finalize the overall documentation report.
- Finalize the individual report.

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the **STUDENT** to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
7. The log sheet is an important deliverable for the research and an important record of a student's organisation and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Subma Thakurathi	Group: 24	Date: 1st April 2026	Meeting No: 4
---------------------------------------	------------------	-----------------------------	----------------------

Project Title: A Web-Based Price Monitoring, Analysis, and Forecasting System for Kalimati Vegetable Market
--

Supervisor's Name: Rukesh Shrestha	Supervisor's Signature: 
---	--

Item for Discussion: (Noted by Student Before Supervisory meeting) - Individual Progress Report - Final guidance and improvements on system design documentation, data pipeline pseudocode, and literary review sections - Report Finalization
--

Record of Discussion: (Noted by Student During Supervisory meeting) - Supervisor checked up on the progress on each individual's contribution. - Log sheets checking. - Comments on the documentation — supervisor provided final remarks on the ETL pipeline design, database schema, and ARIMA forecasting pseudocode sections.

Action List (Noted by Student Next Supervisory meeting) - Incorporate all final supervisor comments; ensure all pseudocode (DataPipeline, AnalyticsEngine, MockDataGenerator) and diagrams are finalized and consistent. - Get the entire report checked by the supervisor and obtain the Turnitin report. - Report Submission.
